EXPERIMENT NO. 6

AIM:
To design the architecture and implement a CNN model for digit recognition applications.

THEORY:
CNN stands for Convolutional Neural Network. It is a type of neural network that is commonly used in computer vision tasks such as image recognition, object detection, and segmentation.
A CNN consists of multiple layers, including convolutional layers, pooling layers, and fully connected layers. The convolutional layers apply a series of filters to the input image to extract features, such as edges and shapes. The pooling layers down sample the output of the convolutional layers to reduce the spatial dimensions of the feature maps. The fully connected layers then classify the features into the desired output classes.
CNNs have revolutionized the field of computer vision and have achieved state-of-the-art performance on various benchmark datasets. They have also been applied to other domains such as natural language processing and audio processing.

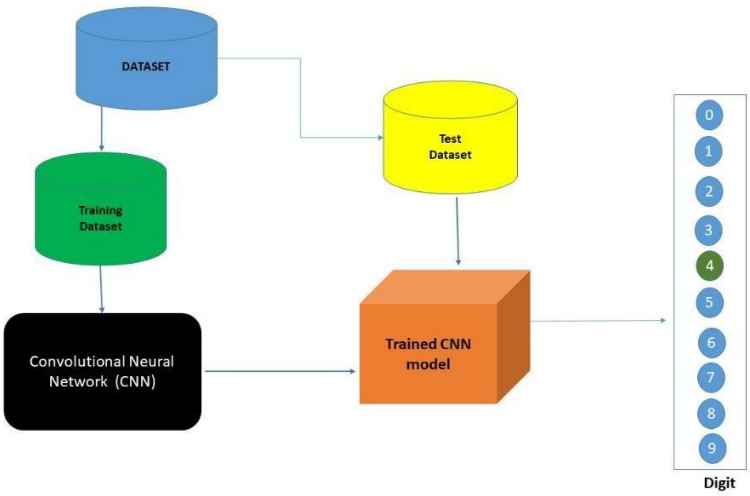


Figure 1: CNN Model For Digit Recognition

CODE:
```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Reshape data to include channel dimension (28, 28, 1)
x_train = x_train.reshape(x_train.shape[0], 28, 28, 1)
x_test = x_test.reshape(x_test.shape[0], 28, 28, 1)
input_shape = (28, 28, 1)
```

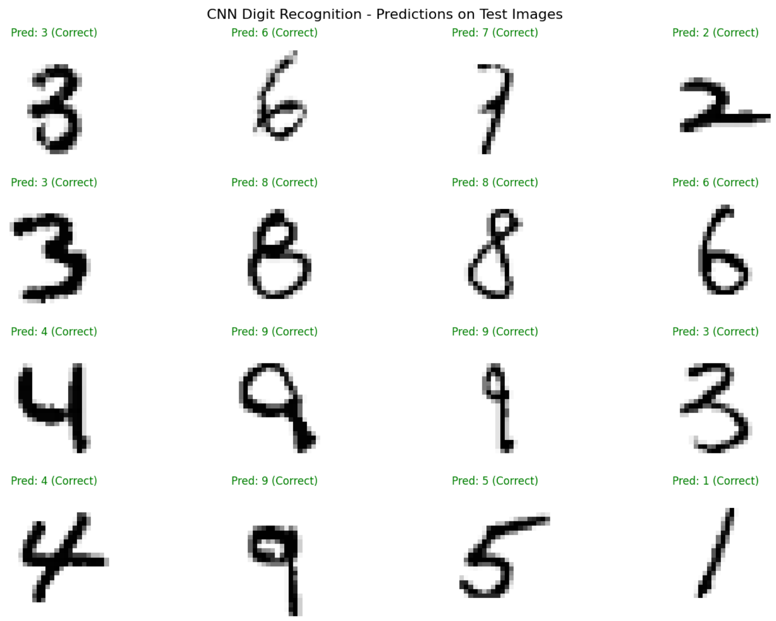
```
# Convert to float32 and normalize to [0, 1]
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255

# Convert labels to categorical one-hot encoding
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

print(f"Training data shape: {x_train.shape}")
print(f"Training labels shape: {y_train.shape}")
print(f"Testing data shape: {x_test.shape}")
print(f"Testing labels shape: {y_test.shape}")

# Build the CNN model
model = tf.keras.models.Sequential([
    # First convolutional layer
    tf.keras.layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=input_shape),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Second convolutional layer
    tf.keras.layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),

    # Flatten and dense layers
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Display model architecture
model.summary()

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(x_train, y_train,
                    batch_size=128,
                    epochs=10,
                    validation_split=0.2,
                    verbose=1)

# Evaluate the model on test data
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
```

```
print(f"\nTest accuracy: {test_acc:.4f}")
print(f"Test loss: {test_loss:.4f}")

# Plot training history
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

# Make predictions on test images and display results
plt.figure(figsize=(15, 10))

# Select test images for prediction
test_indices = [2853, 2000, 1500, 1345, 500, 800, 1200, 3000, 3500, 4000, 4500, 5000, 5500,
6000, 6500, 7000]

for i, image_index in enumerate(test_indices[:16]):  # Display up to 16 images
    plt.subplot(4, 4, i+1)

    # Get the image and make prediction
    test_image = x_test[image_index].reshape(1, 28, 28, 1)
    pred = model.predict(test_image, verbose=0)
    predicted_label = pred.argmax()
    true_label = np.argmax(y_test[image_index])

    # Display the image
    plt.imshow(x_test[image_index].reshape(28, 28), cmap='Greys')

    # Set title with prediction and true label
    if predicted_label == true_label:
        color = 'green'
        title = f'Pred: {predicted_label} (Correct)'
```

```
    else:
        color = 'red'
        title = f'Pred: {predicted_label} (True: {true_label})'

    plt.title(title, color=color)
    plt.axis('off')

plt.suptitle('CNN Digit Recognition - Predictions on Test Images', fontsize=16)
plt.tight_layout()
plt.show()

# Display prediction probabilities for a single image
plt.figure(figsize=(12, 5))

# Choose a test image
sample_idx = 2853
test_image = x_test[sample_idx].reshape(1, 28, 28, 1)
pred_probs = model.predict(test_image, verbose=0)[0]
true_label = np.argmax(y_test[sample_idx])

plt.subplot(1, 2, 1)
plt.imshow(x_test[sample_idx].reshape(28, 28), cmap='Greys')
plt.title(f'Test Image (True Label: {true_label})')
plt.axis('off')

plt.subplot(1, 2, 2)
bars = plt.bar(range(10), pred_probs, color='steelblue', alpha=0.7)
plt.xlabel('Digit Class')
plt.ylabel('Probability')
plt.title('Prediction Probabilities')
plt.xticks(range(10))
plt.ylim([0, 1])
```

OUTPUT:

```
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 53s 126ms/step - accuracy: 0.8769 - loss: 0.3914 - val_accuracy: 0.9754 - val_loss: 0.0852
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 47s 124ms/step - accuracy: 0.9600 - loss: 0.1335 - val_accuracy: 0.9840 - val_loss: 0.0575
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 77s 113ms/step - accuracy: 0.9680 - loss: 0.1019 - val_accuracy: 0.9856 - val_loss: 0.0488
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 82s 112ms/step - accuracy: 0.9736 - loss: 0.0851 - val_accuracy: 0.9878 - val_loss: 0.0415
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 84s 117ms/step - accuracy: 0.9761 - loss: 0.0772 - val_accuracy: 0.9887 - val_loss: 0.0401
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 43s 114ms/step - accuracy: 0.9790 - loss: 0.0677 - val_accuracy: 0.9896 - val_loss: 0.0348
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 45s 120ms/step - accuracy: 0.9806 - loss: 0.0635 - val_accuracy: 0.9895 - val_loss: 0.0365
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 43s 114ms/step - accuracy: 0.9809 - loss: 0.0607 - val_accuracy: 0.9906 - val_loss: 0.0320
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 82s 113ms/step - accuracy: 0.9836 - loss: 0.0533 - val_accuracy: 0.9905 - val_loss: 0.0325
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 45s 119ms/step - accuracy: 0.9836 - loss: 0.0527 - val_accuracy: 0.9916 - val_loss: 0.0316

Test accuracy: 0.9918
Test loss: 0.0242
```



CONCLUSION:

Hence, we can conclude that we can learn and study a CNN model for digit recognition applications.